

faijurrhman /
syzkaller

<> Code

Pull requests

Actions

Projects

Wiki

Security

Insights

syzkaller / docs / linux / setup_linux-host_qemu-vm_arm64-kernel.md



necipfazil and a-nogikh docs/linux/qemu: fix the link to ARM64 toolchain

6b9fb3c · 2 years ago



169 lines (127 loc) · 5.31 KB

Preview

Code

Blame

Raw



Setup: Linux host, QEMU vm, arm64 kernel

This document will detail the steps involved in setting up a Syzkaller instance fuzzing any ARM64 linux kernel of your choice.

Create a disk image

We will use buildroot to create the disk image. You can obtain buildroot from [here](#). Extract the tarball and perform a `make menuconfig` inside it. Choose the following options.

Target options

Target Architecture - Aarch64 (little endian)

Toolchain type

External toolchain - Linaro AArch64

System Configuration

[*] Enable root login with password

() Root password = set your password using this option

[*] Run a getty (login prompt) after boot --->

TTY port - ttyAMA0

Target packages

[*] Show packages that are also provided by busybox

Networking applications

[*] dhcpcd

[*] iproute2

[*] openssh

Filesystem images

```
[*] ext2/3/4 root filesystem
    ext2/3/4 variant - ext3
    exact size in blocks - 6000000
[*] tar the root filesystem
```

Run `make` . After the build, confirm that `output/images/rootfs.ext3` exists.

If you're experiencing a very slow `sshd` start up time with arm64 qemu running on x86, the reason is probably low entropy and it be "fixed" with installing `haveged` . It can be found in the `buildroot menuconfig` :

```
Target packages
  Miscellaneous
    [*] haveged
```



Get the ARM64 toolchain from Linaro

You will require an ARM64 kernel with gcc plugin support. If not, obtain the ARM64 toolchain from Linaro. Get `gcc-linaro-6.1.1-2016.08-x86_64_aarch64-linux-gnu.tar.xz` from [here](#). Extract and add its `bin/` to your `PATH` . If you have another ARM64 toolchain on your machine, ensure that this newly downloaded toolchain takes precedence.

Compile the kernel

Get the source code of the Linux kernel version you want to fuzz, and do the following.

```
$ ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- make defconfig
$ vim .config
```



Change the following options :

```
CONFIG_KCOV=y
CONFIG_KASAN=y
CONFIG_DEBUG_INFO=y
CONFIG_CMDLINE="console=ttyAMA0"
CONFIG_KCOV_INSTRUMENT_ALL=y
CONFIG_DEBUG_FS=y
CONFIG_NET_9P=y
```



```
CONFIG_NET_9P_VIRTIO=y  
CONFIG_CROSS_COMPILE="aarch64-linux-gnu-"
```

```
$ ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- make -j40
```



If the build was successful, you should have a `arch/arm64/boot/Image` file.

Obtain qemu for ARM64

Obtain the QEMU source from git or from the latest source release.

```
$ ./configure  
$ make -j40
```



If the build was successful, you should have a `aarch64-softmmu/qemu-system-aarch64` binary.

Boot up manually

You should be able to start up the kernel as follows.

```
$ /path/to/aarch64-softmmu/qemu-system-aarch64 \  
-machine virt \  
-cpu cortex-a57 \  
-nographic -smp 1 \  
-hda /path/to/rootfs.ext3 \  
-kernel /path/to/arch/arm64/boot/Image \  
-append "console=ttyAMA0 root=/dev/vda oops=panic panic_on_warn=1  
panic=-1 ftrace_dump_on_oops=orig_cpu debug earlyprintk=serial  
slub_debug=UZ" \  
-m 2048 \  
-net user,hostfwd=tcp::10023-:22 -net nic
```



At this point, you should be able to see a login prompt.

Set up the QEMU disk

Now that we have a shell, let us add a few lines to existing init scripts so that they are executed each time Syzkaller brings up the VM.

At the top of `/etc/init.d/S50sshd` add the following lines:

```
ifconfig eth0 up
dhcpd
mount -t debugfs none /sys/kernel/debug
chmod 777 /sys/kernel/debug/kcov
```



Comment out the line

```
/usr/bin/ssh-keygen -A
```



Next we set up ssh. Create an ssh keypair locally and copy the public key to `/authorized_keys` in `/`. Ensure that you do not set a passphrase when creating this key.

Open `/etc/ssh/sshd_config` and modify the following lines as shown below.

```
PermitRootLogin yes
PubkeyAuthentication yes
AuthorizedKeysFile      /authorized_keys
PasswordAuthentication yes
```



Reboot the machine, and ensure that you can ssh from host to guest as.

```
$ ssh -i /path/to/id_rsa root@localhost -p 10023
```



Build syzkaller

Build syzkaller as described [here](#), with `arm64` target:

```
CC=gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-
linux-gnu-g++
make TARGETARCH=arm64
```



Modify your config file and start off syzkaller

A sample config file that exercises the required options are shown below. Modify according to your needs.



```
{
  "name": "QEMU-aarch64",
  "target": "linux/arm64",
  "http": ":56700",
  "workdir": "/path/to/a/dir/to/store/syzkaller/corpus",
  "kernel_obj": "/path/to/linux/build/dir",
  "syzkaller": "/path/to/syzkaller/arm64/",
  "image": "/path/to/rootfs.ext3",
  "sshkey": "/path/to/id_rsa",
  "procs": 8,
  "type": "qemu",
  "vm": {
    "count": 1,
    "qemu": "/path/to/qemu-system-aarch64",
    "cmdline": "console=ttyAMA0 root=/dev/vda",
    "kernel": "/path/to/Image",
    "cpu": 2,
    "mem": 2048
  }
}
```

At this point, you should be able to visit `localhost:56700` and view the results of the fuzzing.

If you get issues after `syz-manager` starts, consider running it with the `-debug` flag. Also see [this page](#) for troubleshooting tips.

